ARCHITECTURESDEMO · .NET 10

Architectures logicielles Fiches mémo

25 démos, une page chacune : l'idée en une phrase, un schéma, les règles qui font l'architecture, quand l'utiliser, les pièges.

Le même métier partout : une petite banque où Alice vire 250 € à Bob.

Le détail (code, tests, déroulements) est dans `Docs/` du dépôt, une page complète par architecture ; la version sans code dans `Docs/pour-tous/`.

Fiches mémo : une page par architecture

Le résumé technique de chaque démo, à relire en deux minutes : l'idée en une phrase, un schéma, les règles qui font l'architecture, quand l'utiliser ou non, les pièges. Pour le code, les tests et les déroulements, chaque fiche renvoie à sa page complète.

🖨️ **Version imprimable** : fiches-architectures.pdf (A4, une fiche par page, 27 pages), régénérée par `python Docs/fiches/pdf/generer-pdf.py` (Chrome ou Edge, connexion à jsDelivr).

📖 Documentation complète : Docs · 📗 Version sans code : pour tous · ★ Concepts transverses (idempotence, outbox, concurrence optimiste, compensation)

① Organisation du code : où mettre quoi, qui dépend de qui ?

#	Fiche	Le cœur de l'idée
0	Couches	Les dépendances descendent vers la base
1	Hexagonale	Ports et adapters ; tout pointe vers le métier
2	Onion	La même règle, avec un centre découpé en couches
3	Clean	Un interactor par cas d'utilisation, un presenter en sortie
4	Cœur fonctionnel	Décider en pur, appliquer dans la coquille
5	Vertical Slice	Un fichier par fonctionnalité
13	Microkernel	Un noyau, des plugins chargés à chaud
20	Pipeline	Des filtres en parallèle, des tuyaux bornés

② Découpage du système : quelles frontières, quel déploiement ?

#	Fiche	Le cœur de l'idée
6	Composants	Une interface publique, le reste `internal`
7	Monolithe modulaire	Des modules par contrats, un seul déploiement
8	Microservices	Un processus et une base par service
12	Serverless	Des fonctions sans état, déclenchées par des événements
14	Saga orchestrée	Un orchestrateur conduit et compense
15	Multi-tenant	Une application, plusieurs clients isolés
16	BFF	Un backend par application cliente
17	Cellules	N copies complètes, rayon d'impact borné
18	Acteurs	État privé, messages un par un, aucun verrou
19	Spatiale	Les données en mémoire, la base en différé
21	Chorégraphie	Chacun réagit aux événements des autres

③ Modélisation : comment représenter le métier et l'état ?

#	Fiche	Le cœur de l'idée
9	CQRS	Un modèle pour écrire, un autre pour lire
10	Event Sourcing	On stocke les événements, l'état se rejoue
11	DDD	La langue du métier, des contextes délimités

④ Migration · ⑤ Exposition · ⑥ Présentation

#	Fiche	Le cœur de l'idée
22	Strangler Fig	Une façade, une route migrée à la fois, réversible
23	REST, gRPC, GraphQL	Ressources, procédures, graphe choisi par le client
24	MVC et MVVM	Une vue jetée à chaque requête, ou un ViewModel vivant

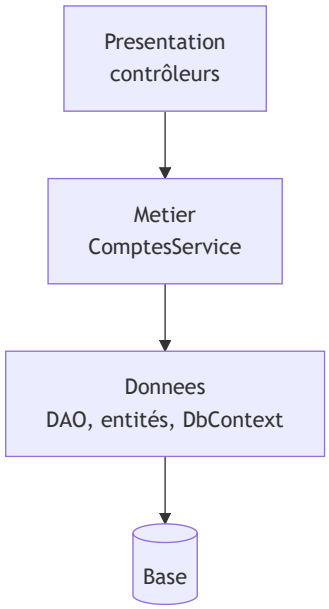
Trois réflexes communs à toutes les fiches

1. **Commencer simple** (Couches, Vertical Slice, Monolithe modulaire) et ne compliquer que face à un vrai problème.
2. **Faire respecter l'architecture** par le compilateur (`internal` , références de projets) ou par un test d'architecture, sinon elle finit contournée.
3. **Dès qu'il y a un réseau ou des messages** : idempotence, outbox, cohérence à terme, compensation (concepts transverses).

Fiche 0 · Couches (N-tiers)

> **En une phrase** : Présentation → Métier → Données ; chaque couche n'appelle que celle du dessous.

Famille : organisation du code · **Origine** : usage courant depuis les années 1990 · **Démo** : `00-Couches` (port 5100) · 📖 Page complète



L'essentiel

- On sépare les responsabilités **techniques**, empilées.
- **Les dépendances descendent jusqu'à la base** : le métier dépend de la couche données (son interface, ses entités).
- Les entités sont **anémiques** : les règles vivent dans les services.
- Garde-fou : références de projets **non transitives** (la présentation ne voit pas les données).

✅ Quand l'utiliser	❌ Signaux pour en sortir
CRUD au métier mince, prototype, équipe débutante	Règles qui se multiplient, à tester sans base
Démarrer vite, avec une structure que tout le monde connaît	Changement de stockage envisagé

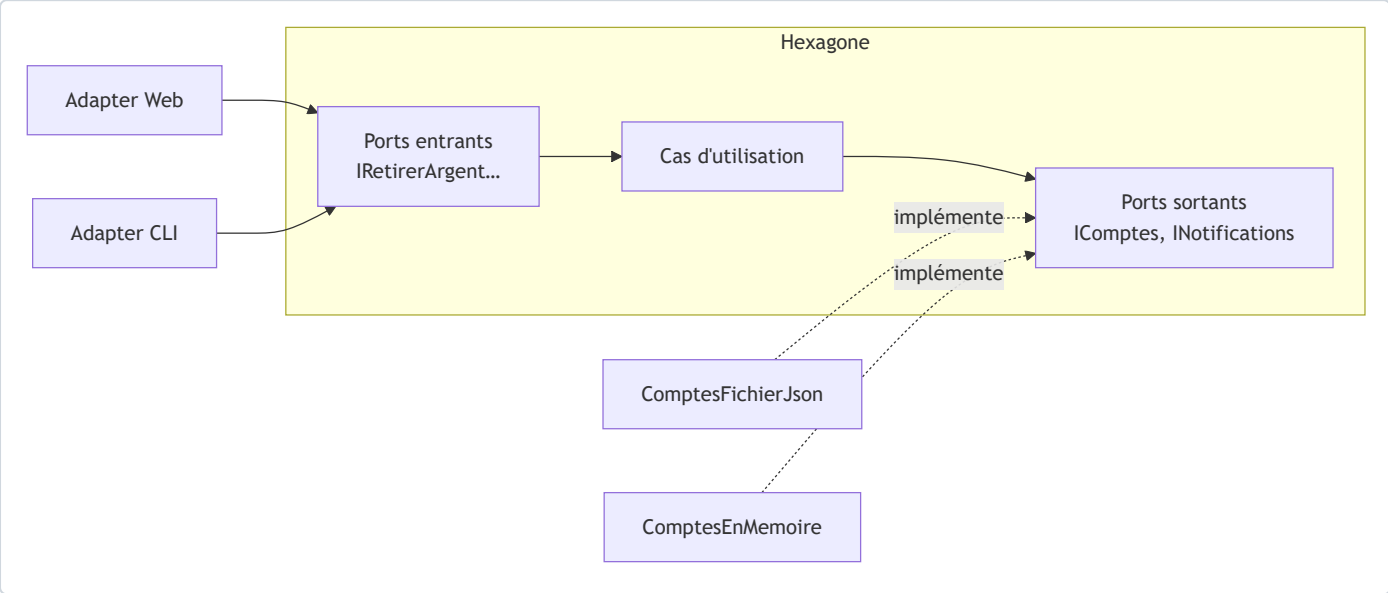
Piège : une fonctionnalité = des modifications dans toutes les couches ; le métier est couplé à la persistance.

Évolutions : inverser les dépendances → Hexagonale, Onion ; ranger par fonctionnalité → Vertical Slice.

Fiche 1 · Hexagonale (Ports & Adapters)

> **En une phrase** : le métier au centre ne dépend de rien ; le monde s'y branche par des **ports** (interfaces) et des **adapters** (implémentations) interchangeables.

Famille : organisation du code · **Origine** : Alistair Cockburn, 2005 · **Démo** : `01-Hexagonale` (port 5101, + une CLI) · 📖 Page complète



L'essentiel

- Intérieur / extérieur plutôt que haut / bas. **Toutes les dépendances pointent vers l'hexagone**.
- Port **entrant** = ce que l'application offre ; port **sortant** = ce dont elle a besoin.
- La *composition root* (`Program.cs`) est le seul endroit qui connaît hexagone et adapters.
- **Tests de contrat** : la même suite tourne sur chaque adapter d'un port.

✅ Quand l'utiliser	❌ Moins utile si
Métier riche, infrastructure susceptible de changer	CRUD simple sans règles
Plusieurs canaux d'entrée (API, CLI, messages)	Un seul adapter par port, pour toujours

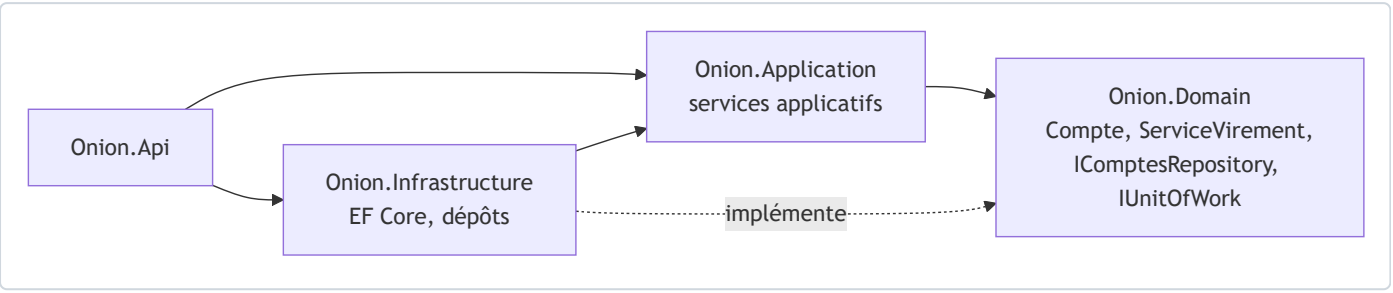
Piège : les ports « passe-plat » qui n'apportent rien ; le centre lui-même n'est pas structuré.

Voisins : même règle de dépendance que Onion et Clean.

Fiche 2 · Onion

> **En une phrase** : des couches concentriques ; le domaine au centre, **toutes les dépendances pointent vers lui**.

Famille : organisation du code · **Origine** : Jeffrey Palermo, 2008 · **Démo** : `02-Onion` (port 5102) · 📖 Page complète



L'essentiel

- Même règle que l'hexagonale, mais le **centre est découpé** : modèle du domaine → services de domaine → services applicatifs.
- Les interfaces d'accès aux données (`IComptesRepository`, `IUnitOfWork`) sont **dans le domaine** ; l'infrastructure les implémente.
- Le service applicatif orchestre : charger, déléguer au domaine, enregistrer (unité de travail, tout ou rien).

✅ Quand l'utiliser	❌ Moins utile si
Métier riche, avec des règles sur plusieurs entités	Petite application : beaucoup de projets et d'interfaces
Besoin d'un centre structuré et d'une unité de travail explicite	Équipe qui ne distingue pas service de domaine et applicatif

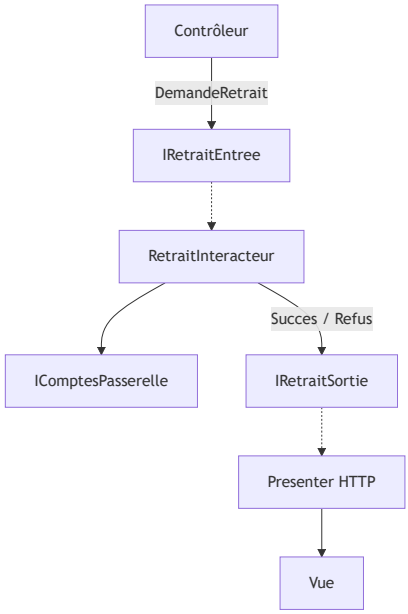
Piège : les règles migrent dans les services → modèle anémique.

Hexagonale ou Onion ? Même règle ; l'une insiste sur les ports autour, l'autre sur les couches au centre. Beaucoup de projets « Clean » .NET sont en fait des Onion.

Fiche 3 · Clean Architecture

> **En une phrase** : un *interactor* par cas d'utilisation, des frontières explicites ; il ne renvoie rien et pousse son résultat vers un *presenter*.

Famille : organisation du code · **Origine** : Robert C. Martin, 2012 · **Démo** : `03-Clean` (port 5103) · 📖 Page complète



L'essentiel

- Quatre cercles : **Entities** → **Use Cases** → **Interface Adapters** → **Frameworks & Drivers**.
- **Règle de dépendance** : un cercle ne connaît que les cercles plus intérieurs.
- Vocabulaire : request / response model, input / output boundary, interactor, presenter, gateway.
- L'inversion de dépendance s'applique aussi à la **sortie** : l'interacteur appelle le presenter via une interface qu'il définit.

✅ Quand l'utiliser	❌ Moins utile si
Plusieurs présentations d'un même cas d'utilisation	API web simple : presenter à état peu naturel en ASP.NET Core
Cas d'utilisation nombreux, à rendre explicites	Équipe qui l'appliquerait de façon dogmatique

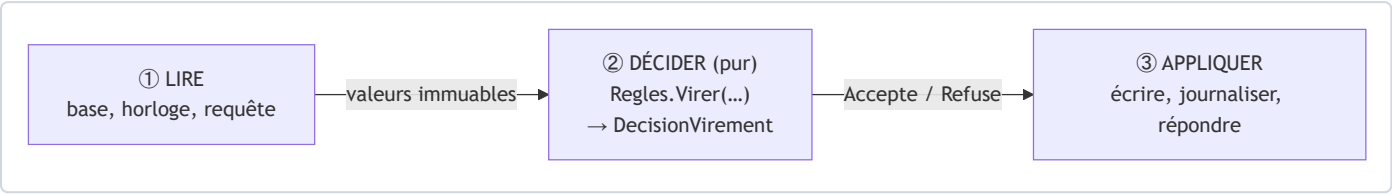
Piège : la **cérémonie** (6 types pour un retrait). En pratique, les modèles .NET (Jason Taylor, Ardalis) gardent les cercles et remplacent le presenter par un `Result<T>`.

Comparer : le même retrait tient en un fichier en Vertical Slice.

Fiche 4 · Functional Core, Imperative Shell

> **En une phrase** : des règles **pures** qui renvoient une décision ; une fine **coquille** autour fait toutes les entrées/sorties.

Famille : organisation du code · **Origine** : Gary Bernhardt, « Boundaries », 2012 · **Démo** : `04-CoeurFonctionnel` (port 5104) · 📖 Page complète



L'essentiel

- Le cœur : pas d'`async`, pas d'interface, pas de base, pas d'horloge. **Des valeurs en entrée, une décision en sortie.**
- La coquille : impérative, mince, **aucune décision**.
- Les décisions sont des **données** : journalisables, rejouables.
- Tests du cœur : valeurs → valeur, **sans aucun mock**, en millisecondes.

✅ Quand l'utiliser	❌ Difficile si
Règles complexes : tarification, éligibilité, plafonds, calculs	La décision exige des lectures en cascade
À l'intérieur de n'importe quelle autre architecture	L'équipe n'a pas l'habitude du style fonctionnel

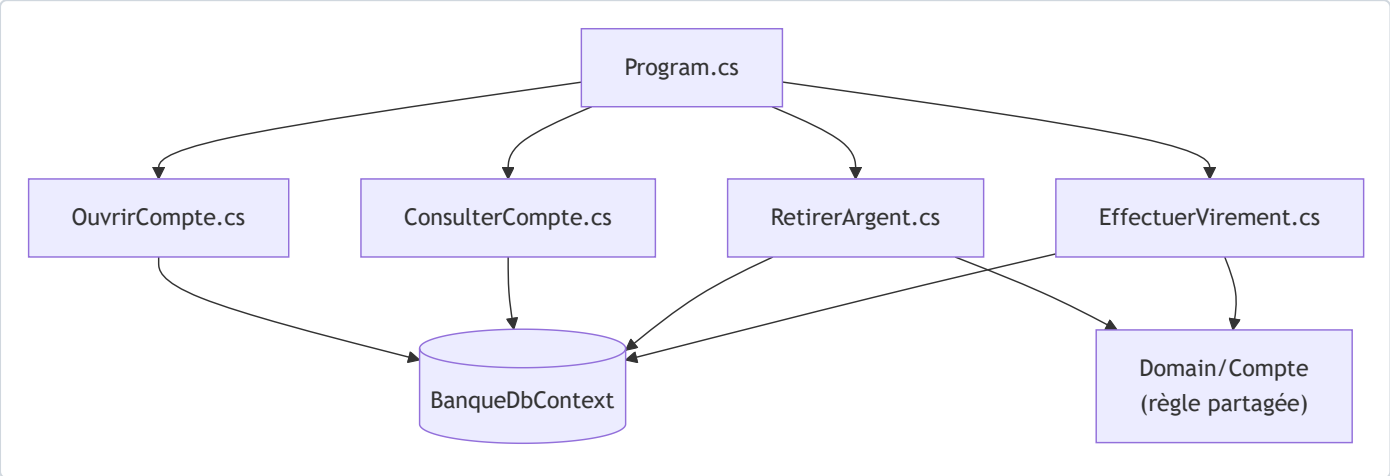
Piège : il faut **tout charger avant** de décider.

Voisins : l'hexagonale poussée à l'extrême ; le même « décider / appliquer » est au cœur de l'Event Sourcing.

Fiche 5 · Vertical Slice

> **En une phrase** : le code rangé **par fonctionnalité** ; chaque tranche contient tout, de la route HTTP jusqu'à la base.

Famille : organisation du code · **Origine** : Jimmy Bogard, vers 2018 · **Démo** : `05-VerticalSlice` (port 5105) · 📖 Page complète



L'essentiel

- Une tranche = une requête ou une commande, dans **un fichier**.
- **Couplage fort dans** la tranche, **faible entre** les tranches.
- Chaque tranche choisit sa technique (projection LINQ, entité, SQL…).
- Pas d'abstraction « au cas où » : le `DbContext` est déjà dépôt + unité de travail. Seules les règles **partagées** remontent dans `Domain/`.

✅ Quand l'utiliser	❌ Moins adapté si
API de gestion, livraison fonctionnalité par fonctionnalité	Changement de base probable (le `DbContext` est partout)
Dans chaque module d'un monolithe modulaire	Équipe peu disciplinée sur le code partagé

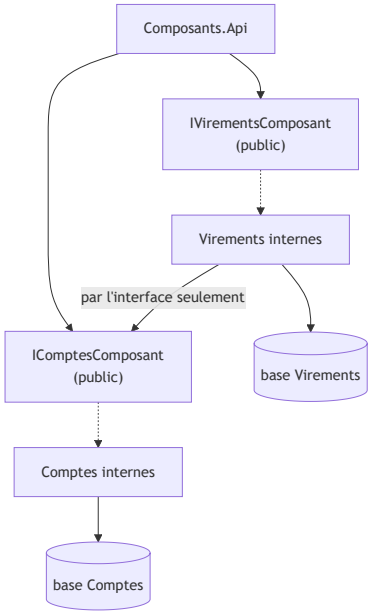
Piège : duplication et handlers obèses si l'on ne remonte jamais rien. (MediatR est payant en entreprise depuis 2025.)

Voisins : l'axe vertical continue avec Composants puis Monolithe modulaire.

Fiche 6 · Package by Component

> **En une phrase** : une **seule interface publique** par composant, tout le reste `internal` ; le compilateur fait respecter l'architecture.

Famille : découpage du système · **Origine** : Simon Brown, « The Missing Chapter » (*Clean Architecture*) · **Démo** : `06-Composants` (port 5106) · 📖 Page complète



L'essentiel

- Constat : si tout est `public`, rien n'empêche un contrôleur d'appeler le dépôt directement.
- Un composant = métier + données derrière une interface ; un raccourci **ne compile pas**.
- **Une base par composant** : Virements ne lit jamais les tables de Comptes.

✅ Quand l'utiliser	❌ Coût
Faire respecter des frontières sans test ni revue	Concevoir de bonnes interfaces publiques demande du soin
Préparer un futur monolithe modulaire	Plus de transaction commune : compensation nécessaire

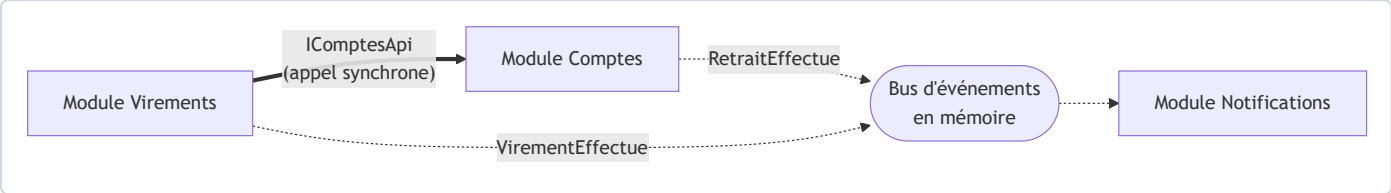
À retenir : une architecture que ni `internal` ni un test d'architecture (NetArchTest, ArchUnitNET) ne protège finit toujours contournée.

Étape suivante : des composants qui possèdent aussi leur API et leurs événements → Monolithe modulaire.

Fiche 7 · Monolithe modulaire

> **En une phrase** : des modules autonomes (API, base, événements) reliés par des **contrats**, déployés ensemble dans un seul processus.

Famille : découpage du système · **Origine** : popularisé notamment par Shopify · **Démo** : `07-MonolitheModulaire` (port 5107) · 📖 Page complète



L'essentiel

- Un module = une mini-application (routes, base, abonnements) branchée d'une ligne : `IModule[] modules = [...]`.
- Règles, vérifiées par tests d'architecture :
  1. on ne référence que les **Contrats** d'un autre module, jamais son implémentation ;
  2. seule la classe `IModule` est publique ;
  3. les contrats ne dépendent d'aucun module ;
  4. **une base par module**.

✅ Quand l'utiliser	❌ Limites
Point de départ recommandé d'un système qui grandit	Pas de mise à l'échelle indépendante des modules
Plusieurs sous-domaines, une seule équipe d'exploitation	Ni transaction ni jointure entre modules

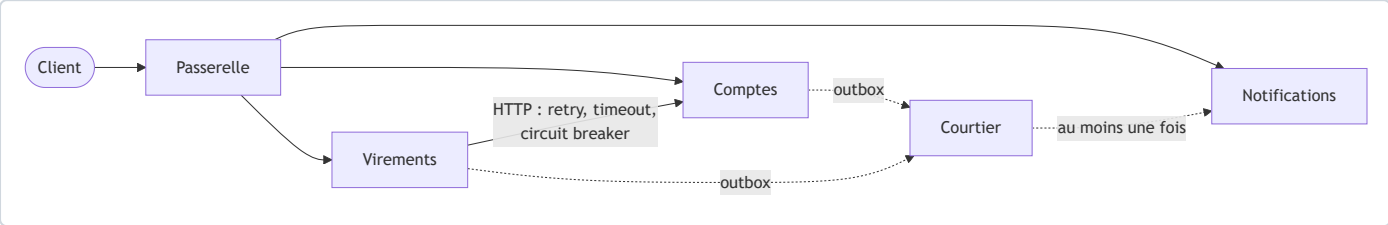
Piège : le bus en mémoire perd les événements en cas de crash → outbox. Une mauvaise découpe coûte cher : trouver les frontières avec le DDD.

Étape suivante : un module à déployer à part → ses contrats deviennent une API réseau → Microservices.

Fiche 8 · Microservices

> **En une phrase** : un **processus par service**, une base chacun, déployable seul ; communication par le **réseau** (HTTP + messages), aucun code partagé.

Famille : découpage du système · **Démo** : `08-Microservices` (passerelle 5200, services 5201–5203, courtier 5209 ; tout démarrer avec `Micro.Lanceur`) · 📖 Page complète



L'essentiel

- La démo 7 coupée en processus : appel de méthode, transaction, événement en mémoire passent par **un réseau qui peut échouer**.
- Le contrat, c'est le **JSON échangé** : aucune référence entre services.
- Mécanismes obligatoires : **idempotence** (`operationId`, `Message-Id`), **outbox**, client résilient, **saga + compensation**, passerelle qui cache les routes `/interne/…`.

✅ Quand l'utiliser	❌ Éviter si
Plusieurs équipes, cycles de déploiement indépendants	Une seule équipe : le monolithe modulaire suffit
Besoins de mise à l'échelle très différents	Pas d'outillage d'exploitation (traces, supervision)

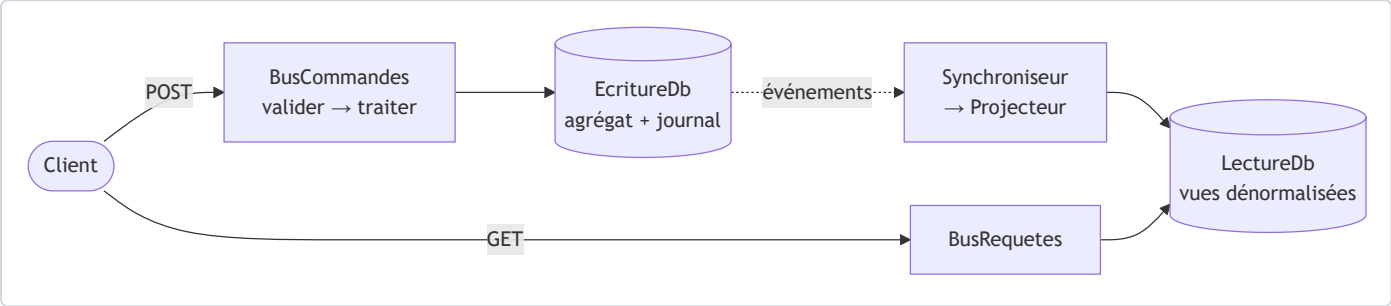
Piège : pannes partielles et cohérence à terme partout ; l'essentiel du code gère les échecs.

En production : RabbitMQ / Azure Service Bus, YARP, .NET Aspire ou Kubernetes, OpenTelemetry. Mécanismes : concepts transverses.

Fiche 9 · CQRS

> **En une phrase** : un modèle pour **écrire** (les règles), un autre pour **lire** (des vues prêtes à afficher), reliés par des événements.

Famille : modélisation · **Origine** : Greg Young, vers 2010, d'après le CQS de Bertrand Meyer · **Démo** : `09-CQRS` (port 5108) · 📖 Page complète



L'essentiel

- Écrire et lire n'ont pas les mêmes besoins : cohérence et invariants d'un côté, rapidité et formes variées de l'autre.
- Une commande ne renvoie rien (ou un identifiant) ; une requête ne modifie rien.
- Les deux côtés **ne se référencent pas** : seul lien, le projet des événements.
- Les vues sont **reconstructibles** à partir des événements.

✅ Quand l'utiliser	❌ Excessif si
Lectures bien plus nombreuses ou très différentes des écritures	CRUD simple
Écrans qui veulent des vues pré-calculées	La cohérence immédiate est exigée partout

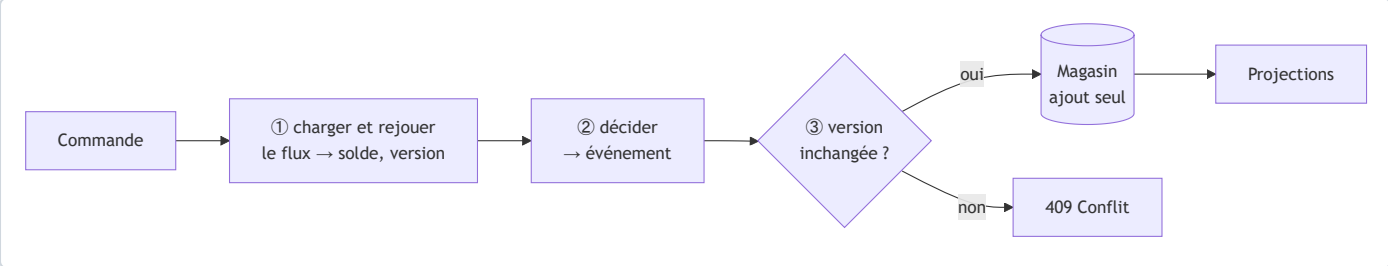
Pièges de vocabulaire : CQRS **n'impose pas deux bases** (séparer les modèles sur une même base suffit souvent) ; CQRS **n'est pas** l'Event Sourcing (ici, l'état est stocké normalement).

Coût : données dupliquées, projections à maintenir, cohérence à terme à expliquer.

Fiche 10 · Event Sourcing

> **En une phrase** : on stocke la **suite des événements**, pas l'état ; l'état se recalcule en les rejouant, et on n'efface jamais rien.

Famille : modélisation · **Démo** : `10-EventSourcing` (port 5109 ; magasin lisible : `Es.Api\evenements.jsonl`) · 📖 Page complète



L'essentiel

- Le solde n'est qu'un **calcul** sur les faits (ouvert, retiré, viré…), comme en comptabilité.
- Agrégat en deux temps : **décider** (règles → événement) puis **appliquer** (événement → état).
- **Concurrence optimiste** par numéro de version ; **instantanés** pour ne pas tout rejouer.
- **Upcasting** des anciennes versions d'événements ; tests **Given / When / Then**.

✅ Quand l'utiliser	❌ Excessif si
L'historique est une donnée métier : banque, compta, assurance, audit	CRUD, ou module sans besoin d'historique
Voyage dans le temps, nouvelles vues rétroactives	Équipe pressée : courbe d'apprentissage réelle

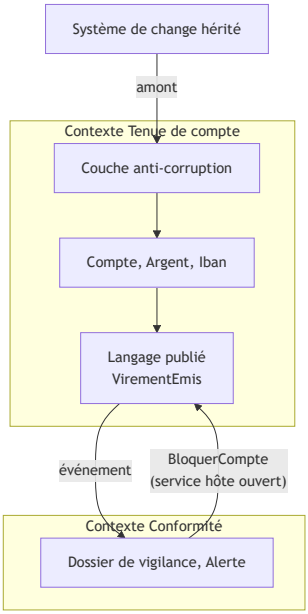
Pièges : on corrige par un événement **compensatoire** ; les événements se **versionnent pour toujours** ; **RGPD** : chiffrer les données personnelles et détruire la clé.

Voisins : presque toujours avec CQRS pour les lectures. En production : EventStoreDB, Marten, Axon.

Fiche 11 · Domain-Driven Design (DDD)

> **En une phrase** : partir du métier, parler **sa** langue, et laisser le modèle guider le code, dans un contexte (tactique) comme entre contextes (stratégique).

Famille : modélisation · **Origine** : Eric Evans, *Domain-Driven Design*, 2003 · **Démo** : `11-DDD` (port 5110, base Onion) · 📖 Page complète



L'essentiel

- **Stratégique** : contextes délimités (un « compte » n'est pas le même en Tenue de compte et en Conformité), carte des contextes, langage publié, couche anti-corruption.
- **Tactique** : langage omniprésent (glossaire `LANGAGE.md` = noms du code), **objets-valeurs toujours valides** (`Iban`, `Argent`), agrégats, fabriques, services de domaine, dépôts, événements de domaine.
- Ce n'est **pas une organisation de dossiers** mais une démarche, applicable avec Hexagonale, Onion ou Clean.

✅ Quand l'utiliser	❌ Excessif si
Métier complexe, experts disponibles	CRUD : beaucoup de cérémonie
Trouver **où couper** modules et services	Pas d'accès aux experts métier

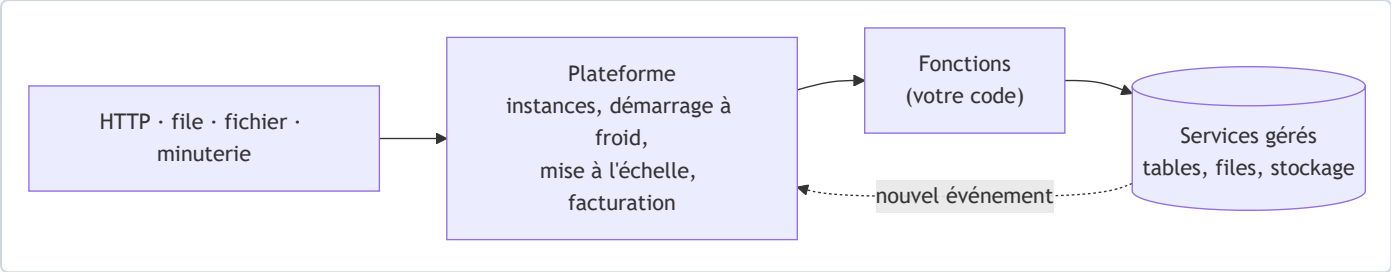
Piège : le réduire à sa boîte à outils tactique ; découper les contextes est difficile et coûteux à corriger.

Liens : contextes → modules (7) ou services (8) ; agrégats → flux d'événements (10).

Fiche 12 · Serverless (FaaS)

> **En une phrase** : des **fonctions sans état** exécutées à la demande quand un **événement** survient, sur des serveurs gérés par le fournisseur, facturées à l'usage.

Famille : découpage du système · **Origine** : AWS Lambda, 2014 · **Démo** : `12-Serverless` (port 5111, plateforme **simulée**) · 📖 Page complète



L'essentiel

- « Serverless » = **sans serveur à gérer**. On écrit des fonctions courtes et un **manifeste** (quelle fonction, quel déclencheur).
- Boucle **événementielle** : une fonction écrit dans une file ou le stockage, ce qui en déclenche une autre.
- État uniquement dans des **services gérés** (concurrence par ETag) ; files avec file *poison*.
- Dans la démo, `Sls.Plateforme` joue le fournisseur ; seul `Sls.Fonctions` serait écrit en vrai.

✅ Quand l'utiliser	❌ Éviter si
Fichiers, messages, webhooks, tâches planifiées	Forte charge constante (coût élevé)
API à trafic irrégulier, « colle » entre services cloud	Latence stricte (démarrages à froid), traitements longs

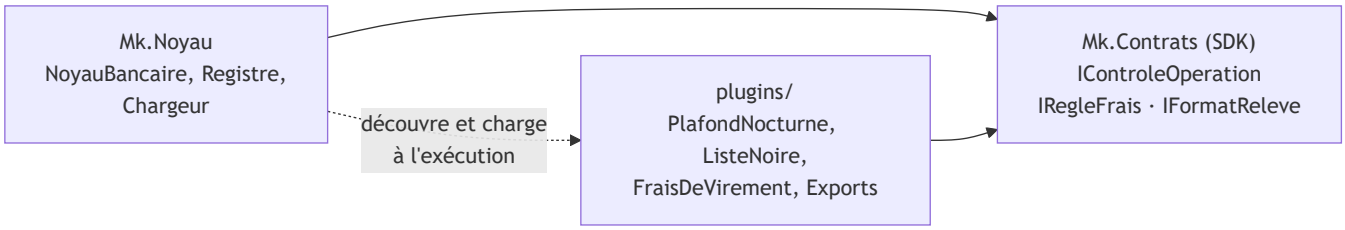
Pièges : **idempotence** obligatoire (livraisons multiples), dépendance au fournisseur, limites de durée et de mémoire.

Pour passer au réel : Azure Functions (isolated worker) ou AWS Lambda, Azurite / LocalStack, Durable Functions / Step Functions pour orchestrer.

Fiche 13 · Microkernel (plugins)

> **En une phrase** : un **noyau minimal** qui définit des points d'extension ; les fonctionnalités variables sont des **plugins** chargés, déchargés et mis à jour à chaud.

Famille : organisation du code · **Exemples** : VS Code, Eclipse, navigateurs, logiciels de paie · **Démo** : `13-Microkernel` (port 5112 ; compiler la solution avant, `plugins/` est une sortie de compilation) · 📖 Page complète



L'essentiel

- Le noyau ne référence **aucun** plugin ; les plugins ne voient que le **SDK**.
- Trois points d'extension : **refuser** une opération, **ajouter des frais**, **produire un relevé** (une DLL peut en contenir plusieurs).
- Un contexte de chargement par plugin : déchargement possible, réglages par `plugin.json`, surveillance du dossier.
- Un plugin défaillant est écarté.

✅ Quand l'utiliser	❌ Attention si
Produit configurable par client, pays, offre	Points d'extension instables : les changer casse tous les plugins
Moteurs de règles, extensions par des tiers	Plugins non fiables : ils tournent **dans le processus**

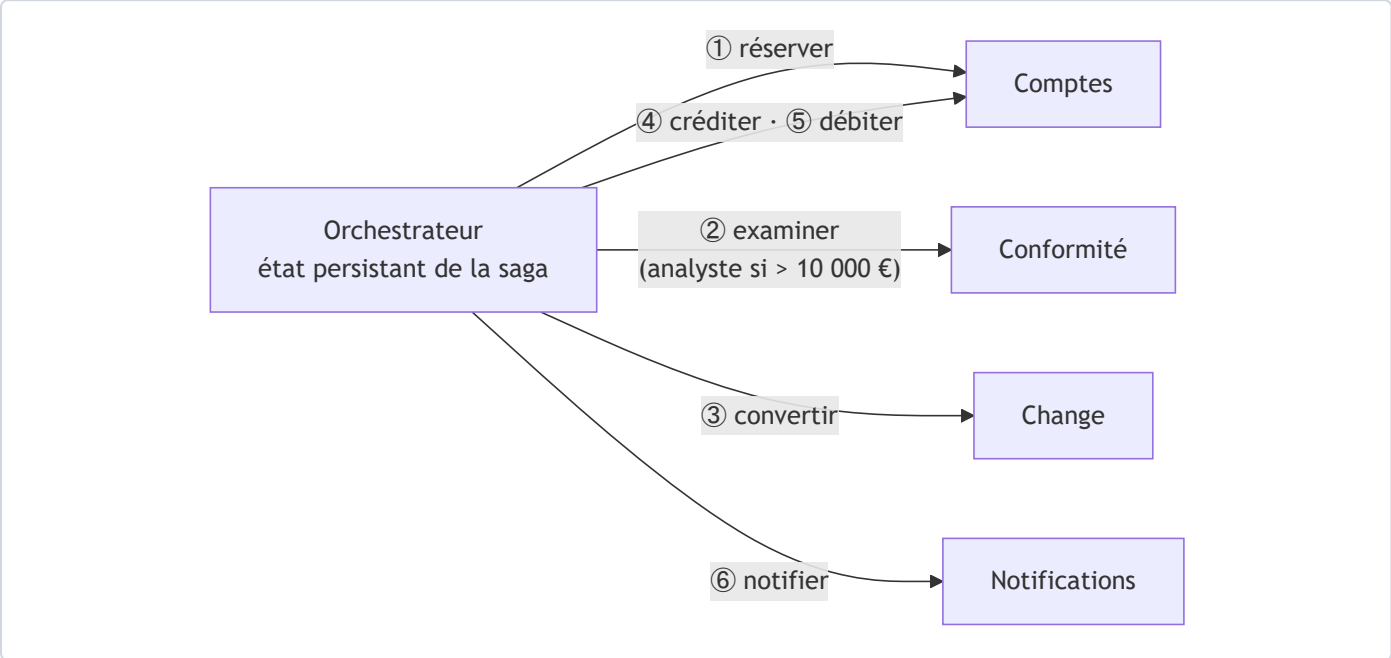
Pièges : copier le SDK dans le dossier du plugin (types incompatibles) ; garder une référence vers un plugin (déchargement impossible) ; le SDK est un contrat public à **versionner**.

En .NET : *McMaster.NETCore.Plugins* encapsule ce mécanisme.

Fiche 14 · Saga orchestrée

> **En une phrase** : un **orchestrateur** pilote un processus réparti sur plusieurs services, mémorise où il en est, réessaie, attend, et **compense** en ordre inverse si besoin.

Famille : découpage du système · **Origine** : sagas, Garcia-Molina et Salem, 1987 · **Démo** : `14-SagaOrchestree` (port 5113) · 📖 Page complète



L'essentiel

- Plusieurs services = pas de transaction unique : des étapes locales + des **compensations**.
- Le processus est **écrit en un seul endroit** ; l'état et l'historique de chaque saga sont consultables.
- Le moteur gère **réessais, attentes (humaines), échéances, compensation** et la reprise après plantage.
- Règle pratique : chorégraphie pour **notifier**, orchestration pour **conduire**.

✅ Quand l'utiliser	❌ Attention à
Processus longs à plusieurs étapes : commande, virement international, souscription	L'orchestrateur connaît tous les participants (couplage)
Attentes humaines, délais, règles d'échec précises	Un composant de plus à rendre hautement disponible

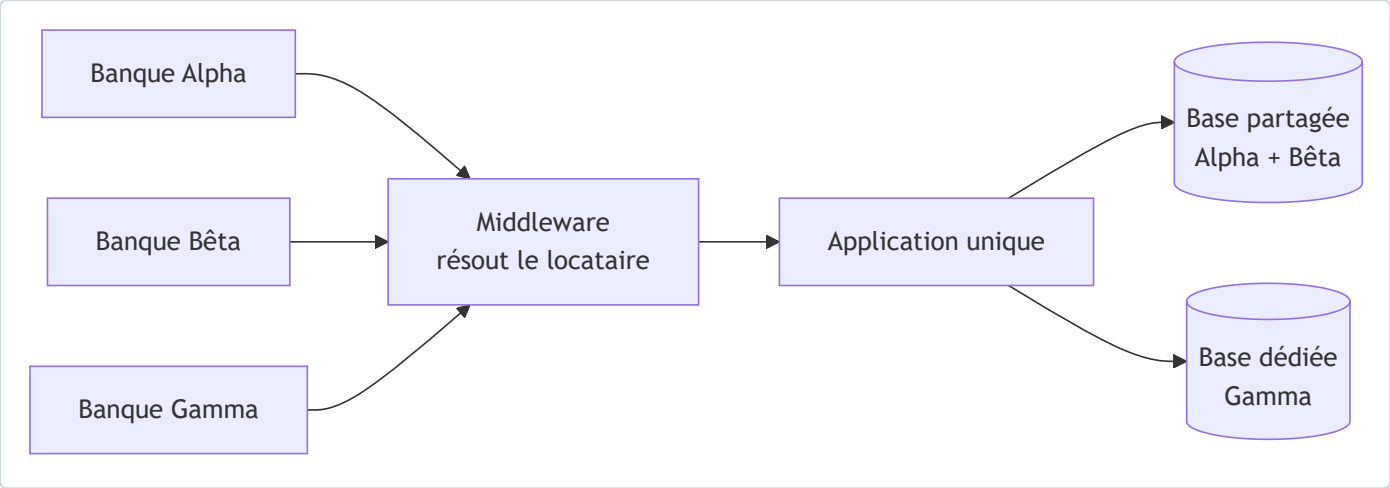
Pièges : participants obligatoirement **idempotents** ; « chef d'orchestre obèse » qui absorbe le métier des participants.

En production : ne pas écrire son moteur → Temporal, Durable Functions, Step Functions, sagas MassTransit / NServiceBus. Comparer avec la Chorégraphie (même virement).

Fiche 15 · Multi-tenant (SaaS)

> **En une phrase** : **une** application, **un** déploiement, pour **plusieurs clients** (locataires), chacun avec ses données, ses règles et ses quotas, sans jamais voir ceux des autres.

Famille : découpage du système · **Démo** : `15-MultiTenant` (port 5114 ; locataire par sous-domaine `beta.localhost:5114` ou en-tête `X-Locataire`) · 📖 Page complète



L'essentiel

- Le locataire est résolu **une fois** (middleware) puis **ambiant** (`ContexteLocataire`).
- Garde-fous : **filtre global** EF Core, **estampillage** automatique à l'écriture, **clé composite** (locataire + id).
- Formules : base partagée par défaut, **base dédiée** pour qui la paie ; quotas contre le **voisin bruyant**.
- `Mt.Locataires` ne contient aucun métier bancaire : le métier reste écrit comme pour une seule banque.

✅ Quand l'utiliser	❌ Coût
Le même logiciel vendu à plusieurs organisations (SaaS B2B)	Une erreur d'isolation expose **tous** les clients
Nouveau client en quelques secondes, une seule version	Migrations et sauvegardes sur N bases dédiées

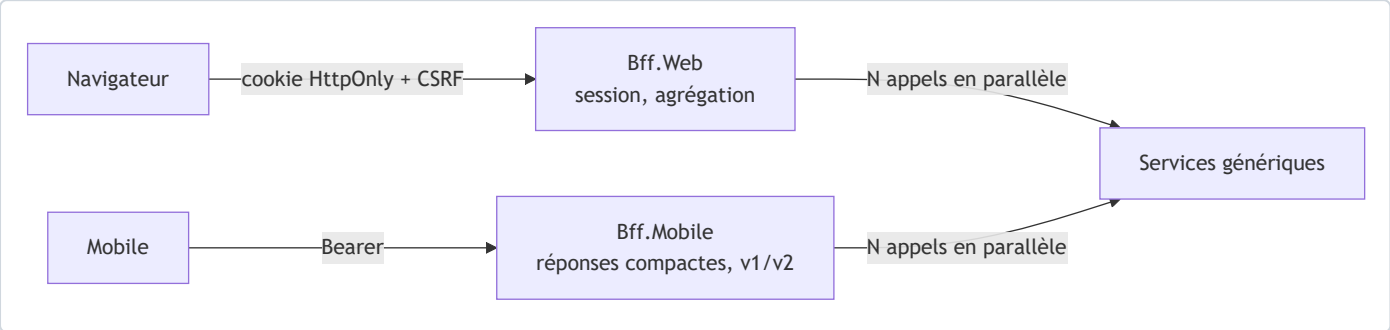
Piège : chaque requête, tâche de fond, cache et journal doit **porter le locataire**.

En production : locataire issu de l'authentification, Finbuckle.MultiTenant, *Row-Level Security* en seconde barrière.

Fiche 16 · Backend for Frontend (BFF)

> **En une phrase** : **un backend par application cliente**, écrit par son équipe, qui assemble les services génériques en réponses taillées pour ses écrans et gère la sécurité à sa façon.

Famille : découpage du système · **Origine** : SoundCloud, popularisé par Sam Newman · **Démo** : `16-BFF` (BFF web 5115, BFF mobile 5116, services 5210 ; `alice / demo`) · 📖 Page complète



L'essentiel

- Sans BFF : le client enchaîne les appels (lent sur mobile) ou les services ajoutent des routes par écran.
- Un appel par écran ; le BFF appelle les services **en parallèle** et dégrade proprement si l'un tombe.
- Côté web, le **jeton reste sur le serveur** (session + cookie HttpOnly).
- Les BFF ne se référencent pas : contrats **recopiés**, seulement les champs utiles (lecteur tolérant).

✅ Quand l'utiliser	❌ Inutile si
Plusieurs applications clientes aux besoins différents	Une seule application, un seul backend (il **est** son BFF)
Application monopage qui ne doit pas manipuler de jetons	

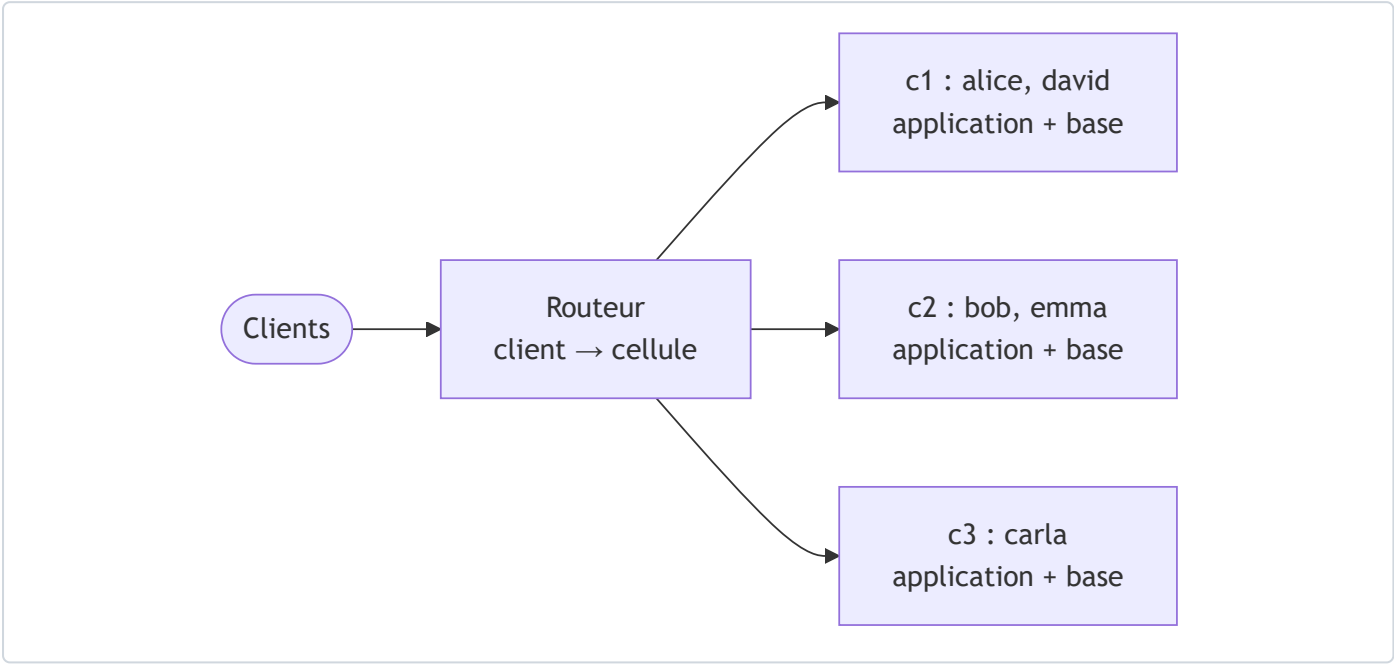
Pièges : y glisser des **règles métier** (elles restent dans les services, qui vérifient aussi les droits) ; sessions à partager entre instances ; duplication voulue, à accepter.

En production : Duende BFF, YARP, OpenID Connect. Alternative : GraphQL devant les applications.

Fiche 17 · Architecture cellulaire (cell-based)

> **En une phrase** : plusieurs **copies complètes et indépendantes** du système, chacune pour une partie des clients ; une panne reste **enfermée dans sa cellule**.

Famille : découpage du système · **Démo** : `17-CellBased` (routeur 5117, cellules 5231–5234 ; tout démarrer avec `Cell.Lanceur`) · 📖 Page complète



L'essentiel

- Objectif : **borner le rayon d'impact** (panne, mauvais déploiement, client gourmand). 50 cellules → un incident touche 2 % des clients.
- Chaque cellule contient **tout** et ne dépend pas des autres.
- Le routeur ne fait que **router** (clé de partition → cellule) ; un plan de contrôle place et déplace les clients.
- Déploiement **par vagues** : la première cellule sert de canari.

	Découpe…	**Une panne touche…**
Microservices	par fonction	une fonction, pour tous
Cellules	par groupe de clients	tout, pour un groupe

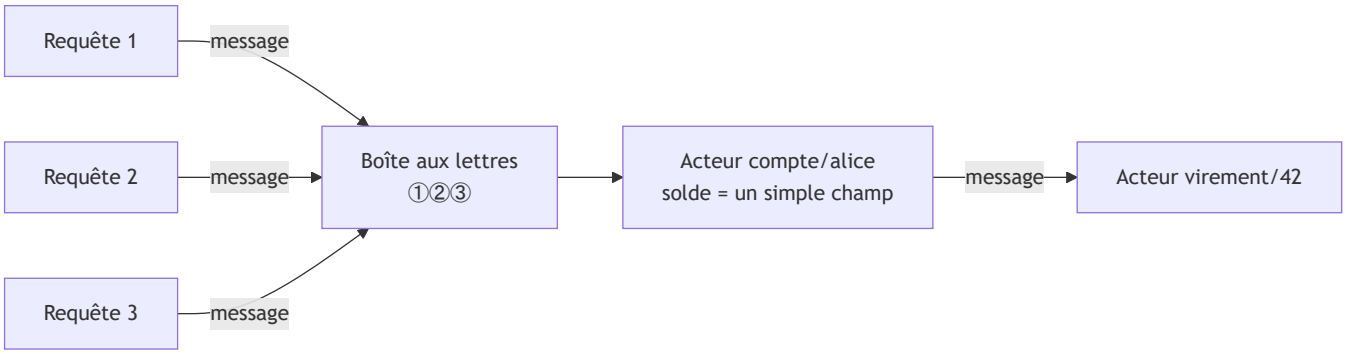
✅ **Quand l'utiliser**	❌ **Excessif si**
Très grande échelle : une panne générale coûte plus que N copies	La plupart des systèmes : zones de disponibilité + déploiements progressifs suffisent
Clé de partition naturelle (le client)	Beaucoup d'échanges entre clients

Pièges : opérations **entre cellules** lentes et à compenser ; clé de partition qui engage pour longtemps ; migrations de clients ; routeur à rendre très disponible.

Fiche 18 · Modèle d'acteurs

> **En une phrase** : des milliers d'acteurs à l'**état privé**, qui ne communiquent que **par messages** traités **un par un** ; aucun verrou, et un acteur qui plante est **remplacé**.

Famille : découpage du système · **Origine** : Carl Hewitt, 1973 ; popularisé par Erlang · **Démo** : `18-ModeleActeurs` (port 5118 ; `/systeme` pour observer et provoquer des pannes) · 📖 Page complète



L'essentiel

- **Chaque donnée a un seul propriétaire** ; la sûreté est structurelle, pas une affaire de discipline.
- Un acteur peut : modifier **son** état, **envoyer** des messages, **créer** des acteurs (ici *virtuels* : on les adresse, le runtime les crée).
- Des acteurs **différents** travaillent en parallèle sur tous les cœurs.
- Une panne : le **superviseur** remplace l'acteur, qui recharge son état durable.

✅ Quand l'utiliser	❌ Moins adapté si
Beaucoup d'entités indépendantes, à l'état souvent modifié en concurrence : jeux, IoT, sessions, comptes	Requêtes transversales sur des données relationnelles
Passer à l'échelle sur plusieurs machines	Besoin de transactions entre entités

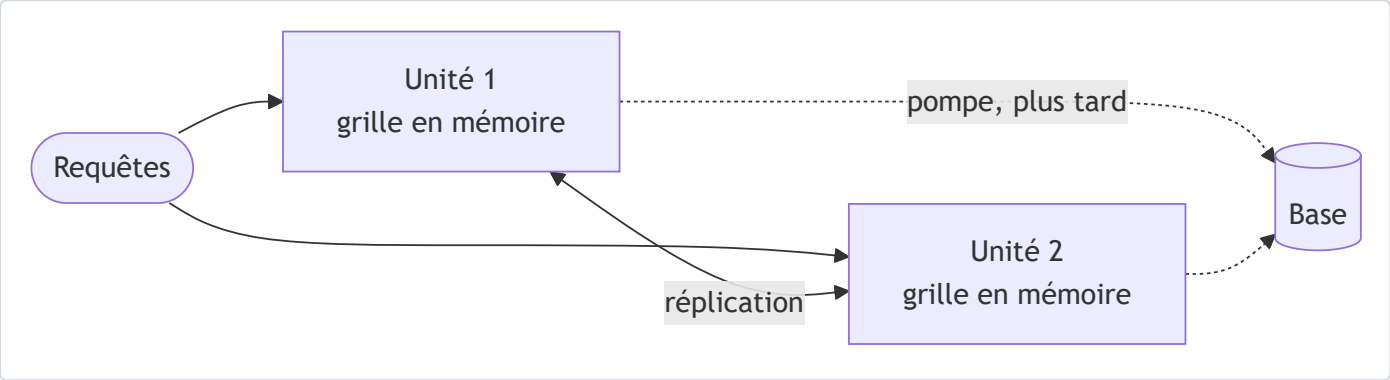
Pièges : pas de transaction entre acteurs (étapes, idempotence, compensation) ; **cycles de demandes = interblocages** ; un acteur très sollicité devient un goulot ; lectures transversales → vue à part.

En .NET : Orleans (acteurs virtuels), Akka.NET ou Proto.Actor (supervision fine).

Fiche 19 · Architecture spatiale (space-based)

> **En une phrase** : la base sort du chemin des requêtes ; chaque **unité de traitement** garde **toutes les données en mémoire**, répliquées entre unités, et la base est mise à jour en arrière-plan.

Famille : découpage du système · **Origine** : *tuple spaces* ; décrite notamment par Mark Richards · **Démo** : `19-SpaceBased` (port 5119 ; `/grille` pour observer et provoquer des pannes) · 📖 Page complète



L'essentiel

- Une requête ne touche que la **mémoire** ; la base n'est plus qu'une **sauvegarde différée**.
- **Élasticité** : une unité démarre en copiant la grille d'une autre ; on en ajoute et retire selon la charge.
- Garde-fous : **versions** sur chaque donnée et **routage par affinité** (même clé → même unité) pour limiter les collisions.
- La base peut tomber sans interrompre le service.

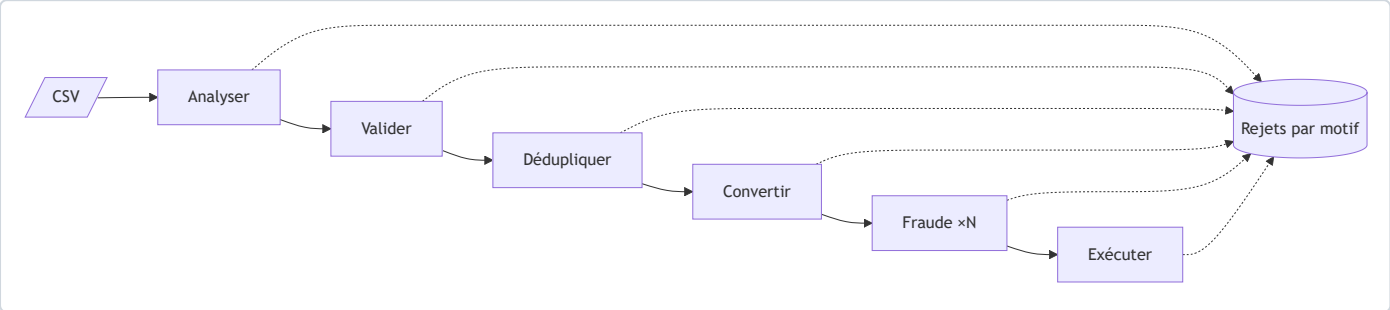
✅ Quand l'utiliser	❌ À éviter si
Pics énormes et imprévisibles : billetterie, enchères, réservations, jeux	Données volumineuses (tout doit tenir en mémoire)
Données écrites surtout par clé	Base qui doit être à jour à chaque instant (compta, rapports temps réel)

Pièges : **collisions** : des écritures acceptées peuvent être perdues ; base en retard pour les autres systèmes ; complexe à tester et à exploiter. Mauvais choix pour de vrais soldes : la démo s'en sert pour **montrer** les collisions.

Fiche 20 · Pipeline (pipes & filters)

> **En une phrase** : des **filtres** indépendants reliés par des **tuyaux bornés**, qui travaillent **en même temps** sur un flux de données, comme une chaîne de montage.

Famille : organisation du code · **Exemples** : shell Unix, compilateurs, ETL · **Démo** : `20-Pipeline` (port 5120 ; `POST /imports` d'un fichier CSV de virements) · 📖 Page complète



L'essentiel

- Un filtre ne connaît ni sa source, ni sa destination : il reçoit, produit ou **rejette** (avec un motif).
- Un tuyau est **borné** : plein, il fait attendre l'étage d'avant (**contre-pression**) → mémoire bornée, quelle que soit la taille du fichier.
- Tous les étages travaillent en parallèle ; pour accélérer, on trouve le **goulot** et on parallélise **cet** étage seulement.
- Types vérifiés à l'assemblage ; mesures par étage.

✅ Quand l'utiliser	❌ Pas pour
Imports / exports, ETL, lots (paie, facturation), flux d'événements, médias	Une application interactive entière
Toute suite d'étapes indépendantes sur des données	Des étapes qui ont besoin de tout le jeu de données

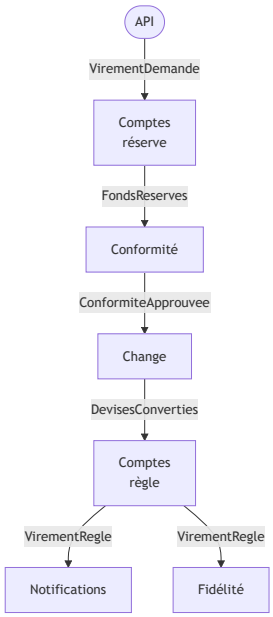
Pièges : le débit est celui du plus lent ; les filtres **à état** (dédoublonnage, exécution) restent uniques et gardent l'ordre ; une panne au milieu laisse un travail partiel → reprise **idempotente**.

À retenir : un pipeline se loge **dans** un service ou un module, organisé lui-même selon une autre fiche.

Fiche 21 · Chorégraphie (événementiel pur)

> **En une phrase** : aucun service ne conduit le processus ; chacun **réagit aux événements** des autres et publie les siens.

Famille : découpage du système · **Démo** : `21-Choregraphie` (port 5121 ; `/bus` et `/chaos` pour observer et casser) · 📖 Page complète



L'essentiel

- Le même virement que la saga orchestrée, mais le processus n'est **écrit nulle part** : il émerge des abonnements.
- Un service ne connaît que des **contrats d'événements** (test d'architecture : aucun service n'en référence un autre).
- Ajouter une réaction = **s'abonner**, sans toucher aux autres.
- « Où en est le virement 42 ? » → une **projection** à construire (le Suivi).

✅ Quand l'utiliser	❌ Préférer l'orchestration si
Réactions à un fait : notifier, facturer, archiver, synchroniser	Beaucoup d'étapes, attentes, délais, règles d'échec
Processus courts, stables, à peu de participants	Besoin de lire et modifier le processus facilement

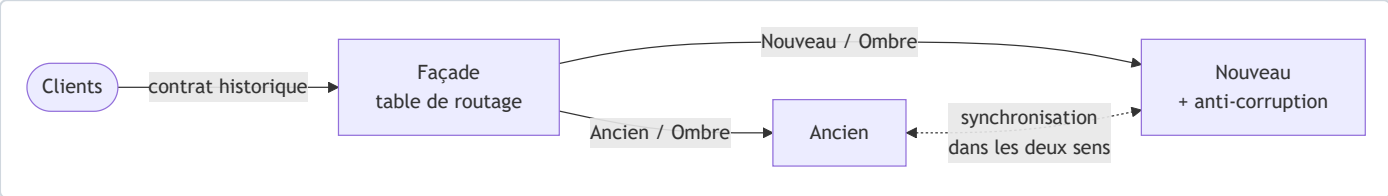
Pièges : compensations et délais **dispersés** ; une panne se traduit par du **silence** (surveiller lettres mortes et virements bloqués) ; point unique de défaillance : le courtier.

Combinaison fréquente : un orchestrateur conduit, et publie des événements que d'autres chorégraphient.

Fiche 22 · Strangler Fig (migration progressive)

> **En une phrase** : une **façade** devant l'ancien système, qui lui retire ses fonctions **une route à la fois** au profit du nouveau, jusqu'à pouvoir l'éteindre.

Famille : migration · **Origine** : Martin Fowler, 2004 · **Démo** : `22-StranglerFig` (façade 5122, ancien 5240, nouveau 5241) · 📖 Page complète



L'essentiel

- Chaque route a un mode : **Ancien**, **Ombre** (l'ancien répond, le nouveau est appelé et comparé), **Nouveau**.
- Table de routage modifiable **à chaud** : chaque pas s'annule **en une requête** ; ce qui n'est pas déclaré reste à l'ancien.
- Le nouveau a son propre modèle et sa base, derrière une **couche anti-corruption**, et reproduit le contrat historique à l'identique.
- Aucun projet ne référence l'ancien : le jour venu, on supprime son projet et l'anti-corruption.

✅ Quand l'utiliser	❌ Moins utile si
Système en production qu'on ne peut ni arrêter ni geler	Petit système réécrit en quelques semaines
Fonctions découpables en routes, écrans ou messages	Aucun point d'entrée à intercepter

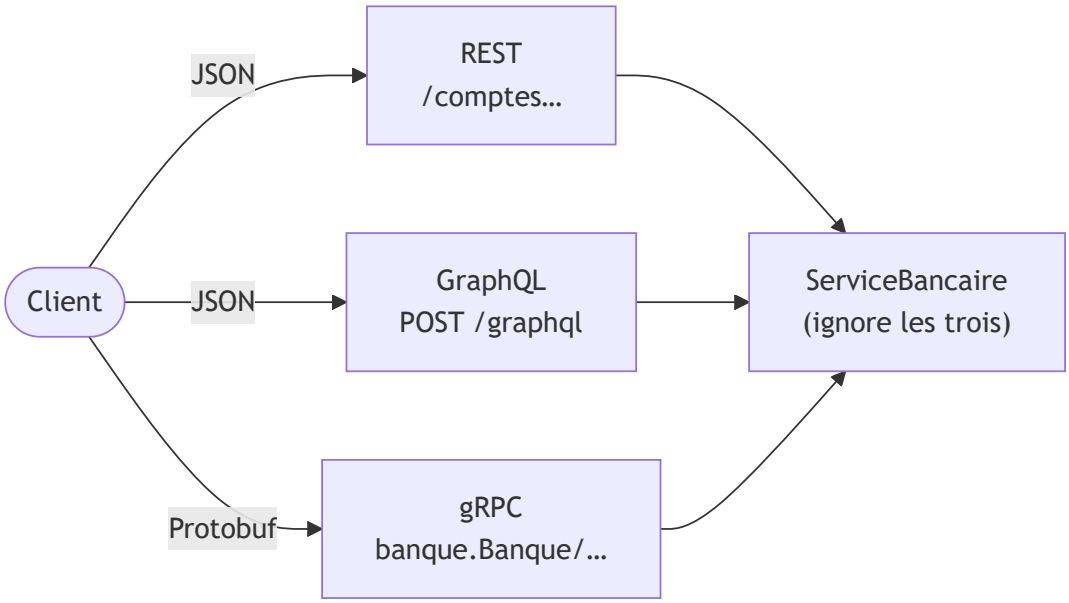
Pièges : synchronisation (boucles, pannes, retard) ; basculer les données **par blocs cohérents** ; contrat hérité à reproduire, bizarreries comprises ; une migration arrêtée à 80 % = le pire des deux mondes.

En production : YARP, Debezium, Scientist / Diffy pour l'ombre, indicateurs de fonctionnalités.

Fiche 23 · REST, gRPC, GraphQL

> **En une phrase** : le même métier exposé en **ressources** (REST), en **procédures** (gRPC) et en **graphe que le client interroge** (GraphQL).

Famille : exposition · **Démo** : `23-RestGrpcGraphQL` (REST et GraphQL 5123 en HTTP/1.1, gRPC 5250 en HTTP/2 ; Protobuf, gRPC et GraphQL écrits à la main) · 📖 Page complète



L'essentiel

	REST	**gRPC**	**GraphQL**
On manipule	des ressources + verbes HTTP	des procédures	un graphe de types
Contrat	implicite ou OpenAPI	`.proto`, code généré	schéma SDL, validé avant exécution
Forme de la réponse	choisie par le serveur	choisie par le serveur	**choisie par le client**
Format / transport	JSON, tout HTTP	binaire, **HTTP/2** obligatoire	JSON, `POST`
Erreurs	statut HTTP + `problem+json`	statut gRPC dans les trailers	`200` + `errors`
Cache HTTP · flux · navigateur	✅ · ❌ · ✅	❌ · ✅ · ❌ (gRPC-Web)	❌ · abonnements · ✅

Quand choisir quoi

- **REST** : API publique ou partenaire, ressources simples, cache.
- **gRPC** : entre services internes ; débit, latence, flux, contrats stricts entre équipes.
- **GraphQL** : clients nombreux aux écrans riches ; un aller-retour par écran, à la place d'un BFF par application.

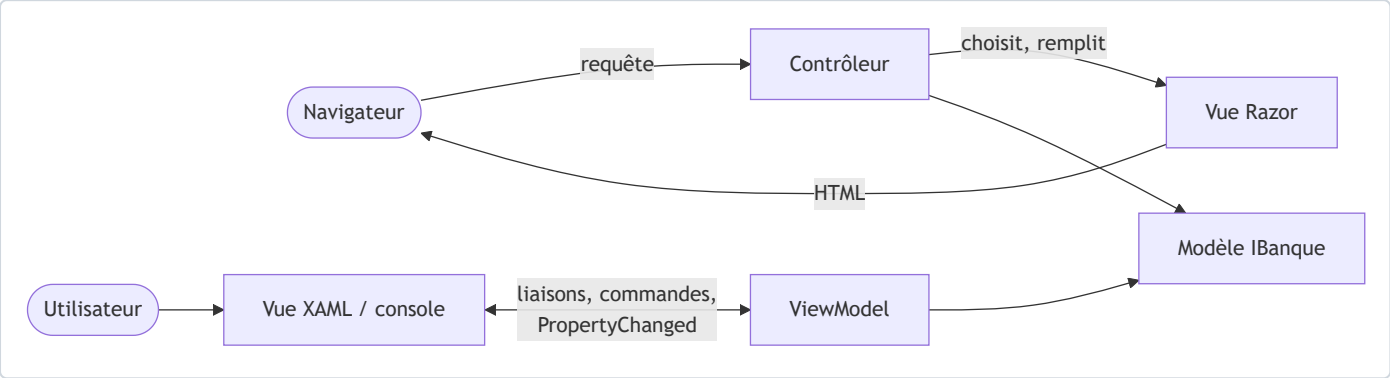
Pièges : REST → allers-retours et sur-récupération ; gRPC → illisible sans outil, pas de navigateur ; GraphQL → **N+1 caché** (DataLoader), requêtes à **borner** (profondeur, coût), cache HTTP perdu.

À retenir : le style se choisit **frontière par frontière** ; les trois cohabitent souvent.

Fiche 24 · MVC et MVVM

> **En une phrase** : le même écran en **MVC** web (un contrôleur reçoit la requête, choisit une vue, jetée après rendu) et en **MVVM** (un ViewModel vivant que la vue observe par liaisons).

Famille : présentation · **Démo** : `24-MvcMvvm` (site MVC 5124 ; vues WPF, Windows seulement, et console sur les mêmes ViewModels) · 📖 Page complète



L'essentiel

	MVC	**MVVM**
Reçoit l'action	le contrôleur	la vue, qui la transmet par liaison ou commande
Qui connaît qui	le contrôleur choisit la vue	la vue connaît le ViewModel, **pas l'inverse**
Durée de vie de l'écran	**une requête**	**tant que la fenêtre est ouverte**
Validation	après l'envoi (ModelState)	**à chaque frappe**
Testable sans écran	la décision du contrôleur	**tout le comportement** de l'écran

- Le modèle ignore les écrans (test d'architecture) ; les ViewModels ignorent WPF (autre test).
- Les liaisons XAML ne sont pas compilées : un test **lit le XAML** pour les vérifier.
- Vocabulaire piège : les « view models » d'ASP.NET MVC sont de simples sacs de données, pas des ViewModels MVVM.

✅ **MVC**	✅ **MVVM**
Site rendu côté serveur, référencement, tenue en charge	Bureau (WPF, WinUI, Avalonia), mobile (MAUI)

Pièges : MVC → une page par étape, état dans des champs cachés. MVVM → abonnements qui fuient, **réponses asynchrones périmées**, fil d'exécution de l'interface, cérémonie (CommunityToolkit.Mvvm aide).

À retenir : MVC et MVVM ne sont que la **couche de présentation** ; le modèle derrière suit n'importe quelle autre fiche.